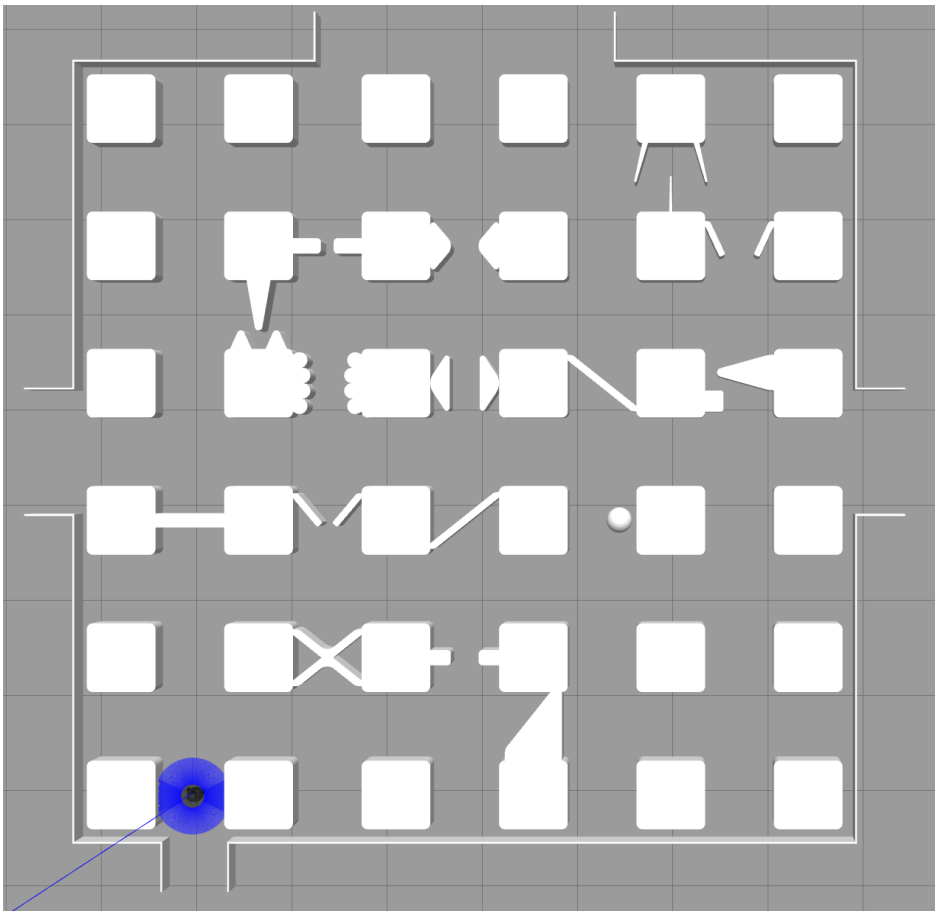
Project Work

Maze Track

Task: write a python ROS node that navigates the TurtleBot through the provided maze track without hitting any walls.



The goal of the project work is to apply the vehicle-control information you have learned during the course, as well as encourage creative thinking and problem-solving. The maze track challenges the vehicle control implementation beyond simple lane following, rudimentarily simulating difficulties that vehicle perception, path planning and control encounter: The 5 by 5 node like structure provides challenges for the robot to detect whether or not the path from one node to another is passable and if so, it needs to have a good control logic to avoid the obstacles in the path. If the path between the obstacles are not passable then the robot needs to create a new route to the final goal using the A* algorithm. You are free to use any control, perception and path planning strategy for solving the task.

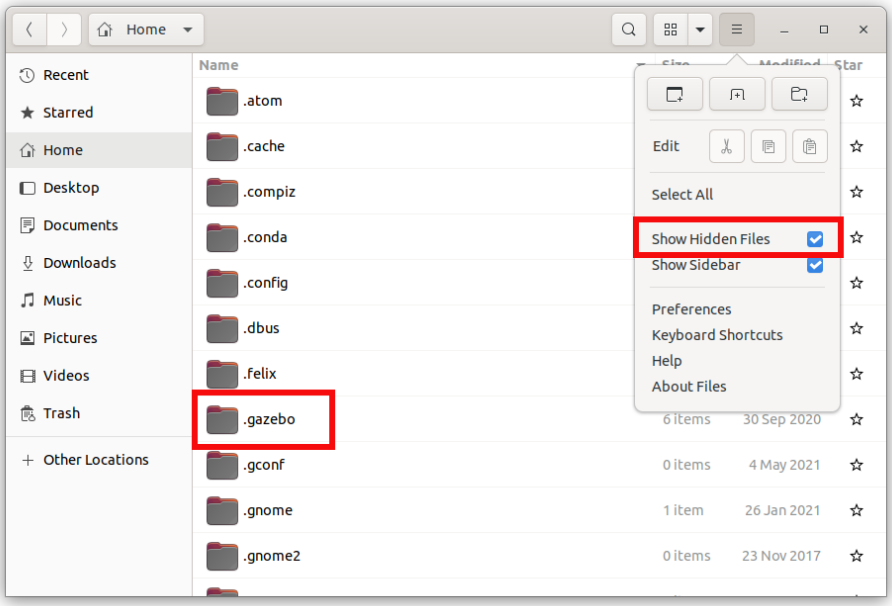
1 Environment Launch Process

On the MyCourses page of the exercise, the following files are provided:

- **project_track.zip**: A 3D model of a custom track that should be placed in the `.gazebo` folder.
- **project_track_no_obs.zip**: A 3D model of a custom track without obstacles, also to be placed in the `.gazebo` folder.
- **vmc2_project**: Contains a launch file that starts the relevant nodes to initiate the Turtlebot simulation within the custom world. It includes both obstacle and obstacle-free worlds, as well as the `navigator.py` file in the `src` folder.

1. Extract the folders **project_track.zip** and **project_track_no_obs.zip** into **`~/.gazebo/models/`**.

   The `.gazebo` folder is hidden; see the figure below for navigation tips.



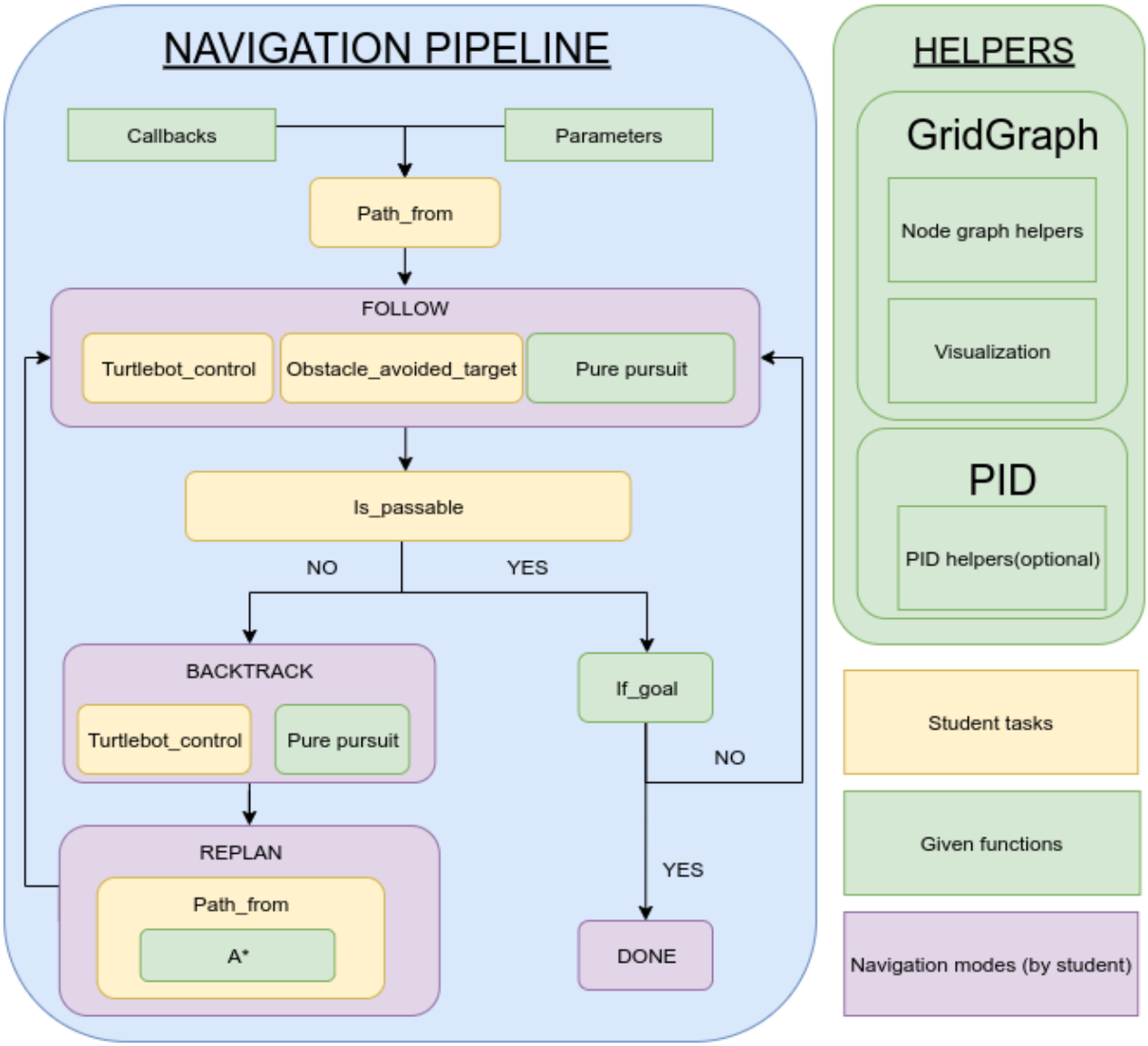
2. Unzip the **vmc2_project** package into your workspace.
3. Build and source your workspace.
4. Launch Gazebo with the following command:

```
ros2 launch vmc2_project project.launch.py
```

You should now see the Turtlebot in the maze. The project must be carried out in this environment.

Getting started

An overview of the navigation framework, including the tasks which you must implement, is shown in the figure below.



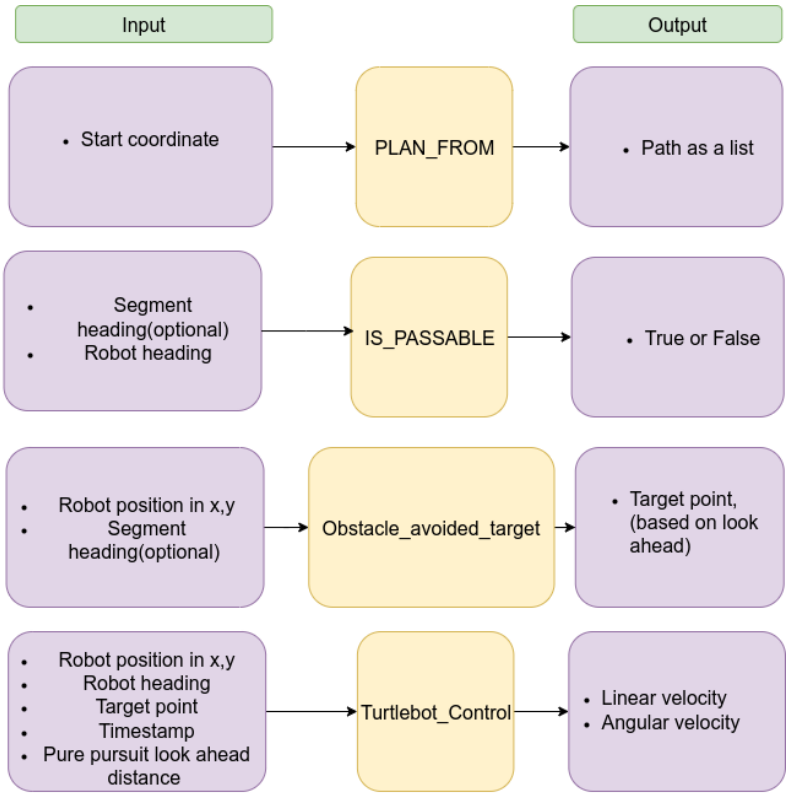
Students must complete three main components:

- Perception: determines whether the path between nodes is passable or blocked.
- Control: ensures the robot follows the planned path reliably without hitting any walls or other obstacles.
- Path planning: generates the best route to the goal point and updates it when a blockage is detected

Using these components, students should implement a navigation logic with four states:

- *FORWARD*: the default state that handles control and perception
- *BACKTRACK*: returns the robot to the latest (previous) visited node
- *REPLAN*: invokes the path-planning module to compute a new route when the current one is blocked
- *DONE*: indicates that the robot has reached its final destination

Figure bellow shows the input and output of the functions that need to be implemented by students.



1.1 General information

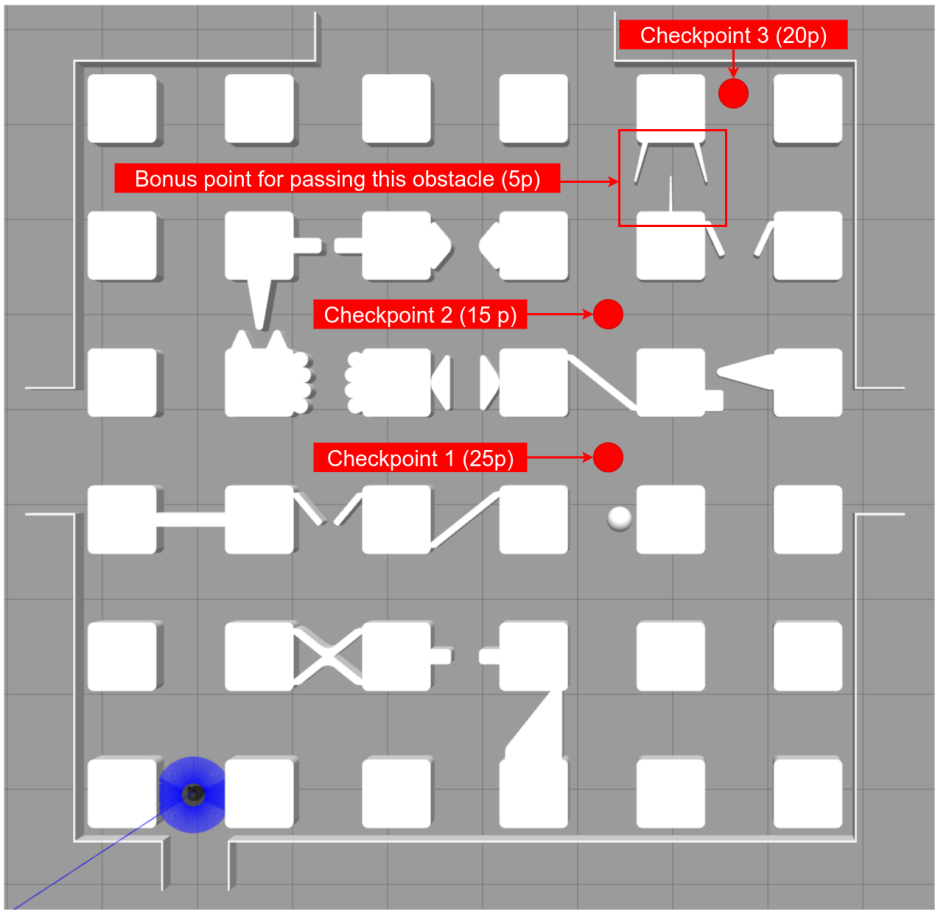
- You may add your own functions (not required) such as helper functions if needed, but please refrain from removing or modifying code provided in the template. If in doubt, always ask!
- If you teleport the TurtleBot, update the start position (`self.start_pos`) to the same value before running the navigator.
- The PID class is optional; use it only if your controller requires PID-based control.
- You may remove edges temporarily for testing, but any final solution that relies on removed edges is invalid.
- Start with the obstacle-free map, then proceed to the map with obstacles.
- At each node, consider rotating the TurtleBot in place to align with the next direction of travel before initiating linear motion.
- **No hard-coding or map prerecording**: Do not embed map-specific information in your code (e.g., memorizing a particular maze or fixed coordinates). Your solution must generalize to, for instance, different obstacle configurations of the maze.
- If anyone wants to use the Turtlebot's dimensions as input to your code (e.g. for the obstacle avoidance logic), that is allowed. The value which is probably most relevant is its **width** which is **0.178 m**
- **Reliability (repeatability):** Your solution should be reliable. Teaching assistants will test it over multiple runs, and it should succeed consistently (e.g., at least two out of three trials or an equivalent success rate).
- **Passing threshold:** 30 points.

Grading

In order to get full points (75p) for the project work, the following requirements have to be met:

- Your implementation must be capable of driving the Turtlebot through the maze from beginning to end without hitting any walls. (60p)
- The Turtlebot must drive through the entire track in less than 2 minutes and 30 seconds (sim time). (10p)
- You must present an overview of your solution at the Project Work Gala. (5p)

The race track is divided into "checkpoints", which determine your grade if your solution is capable of only partially passing through or detecting a blocked passage. Reaching the checkpoints grants you 20 points each, with a total of 3 checkpoints. The checkpoints are highlighted in a figure below. To prevent bottlenecks caused by getting stuck at a certain part of the track, you are free to skip parts you find difficult by "teleporting" to the checkpoints. Any checkpoints you teleport to do NOT naturally grant you the points for reaching them. Bash-commands for teleporting the Turtlebot to each checkpoint are provided at the end of the document.



Note that the completed checkpoints must be reached with a single control solution, the one you submit implemented in your navigator.py-node. You are NOT allowed to change the control (modify code, different PID parameters, etc.), between teleporting the Turtlebot to different checkpoints.

In addition to pure results, solid effort is also taken into consideration in the grading. If you are unable to reach some checkpoint, you can document your tested control approaches and the reasoning behind them. This will award you some points for the checkpoint, based on the clarity of the documentation and logic of the control approach.

Below there is a concrete example to clarify the grading process:

Example project work grading

- The Turtlebot is capable of driving from the beginning to checkpoint 1 without hitting any walls. (25)
- The solution is presented at the Project Work Gala. (5p)

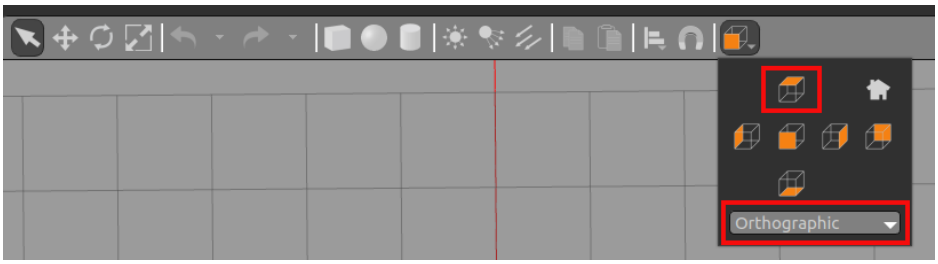
Final grade for the Example project work is 30/75 points. 1 checkpoint were reached, and the work was presented at the Gala getting the passing grade.

Project work gala presentation

The project work presentation has a time limit of five minutes. The presentation should consist of around 3-5 slides, which give a short overview of the selected control , perception and path finding, results, and observations of the control performance (pros and cons). The presentation is given at the Gala, location and date can be checked from MyCourses.

Project documentation

Document your results by recording your screen while the Turtlebot drives through the track. If you are only completing certain checkpoints from different parts of the track, you can record multiple videos, each highlighting the Turtlebot passing the specific checkpoints. If your Turtlebot is capable of driving through the entire track in under 2 minutes and 30 seconds (sim time), make sure that the Gazebo "sim time" is clearly visible in the video to get the points for your accomplishment. For clearer videos, please switch the Gazebo view to "top-view" and "orthographic" (see figure below).



You can use any tool you want for recording the screen. The Ubuntu desktop has a built in screen-recording tool, which can be initiated and finished via the same keyboard shortcut: **ctrl-shift-alt-r**. An orange dot should appear in the top right corner of your desktop, indicating that the recording is in progress. Note that the default maximum length of the recording is 30 s. Before recording, this maximum length should be extended to for example 600 s.

```
$ gsettings set org.gnome.settings-daemon.plugins.media-keys
↪  max-screencast-length 600
```

Note further that the recording tool can be opened from your local desktop as well instead of the VDI so that the recording will not cause jitters in the actual simulation.

In addition to the video, you must submit your **vmc2_project** package containing your **navigator.py** implementation. The submitted **navigator.py** implementation must be capable of replicating the results presented in your video(s)!

If you wish to gain points for effort put into checkpoints you were unable to clear, you must submit clear documentation showing your experimentation process. This documentation should contain written/graphical explanations of the tested control approaches in PDF-format, as well as screen recordings highlighting the results achieved with the tested approaches.

Deliverables

Submit a single zip-file, containing the following documents

1. The **vm2c_project** package, containing your **navigator.py** implementation.
2. Video(s) showing the Turtlebot clearing the maze/checkpoints
3. Optional: documentation highlighting the effort you've put into checkpoints that you were unable to reach.

Commands for teleporting the Turtlebot

Before teleporting the Turtlebot, remember to shut down your control node (**navigator.py**). **Copy-pasting the below commands will likely result in error** due to incorrect indentation. Copy-pastable versions of the commands are provided in the "teleportation_commands_copyable.txt"-file, provided in MyCourses.

Below are teleportation commands for different checkpoints on the track (make sure you change your **start_point** variable in the code to the same value too!)

Teleport to checkpoint 1:

```
$ ros2 service call /set_entity_state gazebo_msgs/srv/SetEntityState "{state: {name: 'burger',
  pose: {position: {x: 3.560, y: -4.272, z: 0.0},
         orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}},
  reference_frame: 'world'}}"
"
```

Teleport to checkpoint 2:

```
$ ros2 service call /set_entity_state gazebo_msgs/srv/SetEntityState "{state: {name: 'burger',
  pose: {position: {x: 4.984, y: -4.272, z: 0.0},
         orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}},
  reference_frame: 'world'}}"
"
```

Teleport to checkpoint 3:

```
$ ros2 service call /set_entity_state gazebo_msgs/srv/SetEntityState "{state: {name: 'burger',
  pose: {position: {x: 7.120, y: -5.696, z: 0.0},
         orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}},
  reference_frame: 'world'}}"
"
```

You can teleport your robot to any node you like for your testing purpose! See the Figure below for the coordinates of the nodes.

